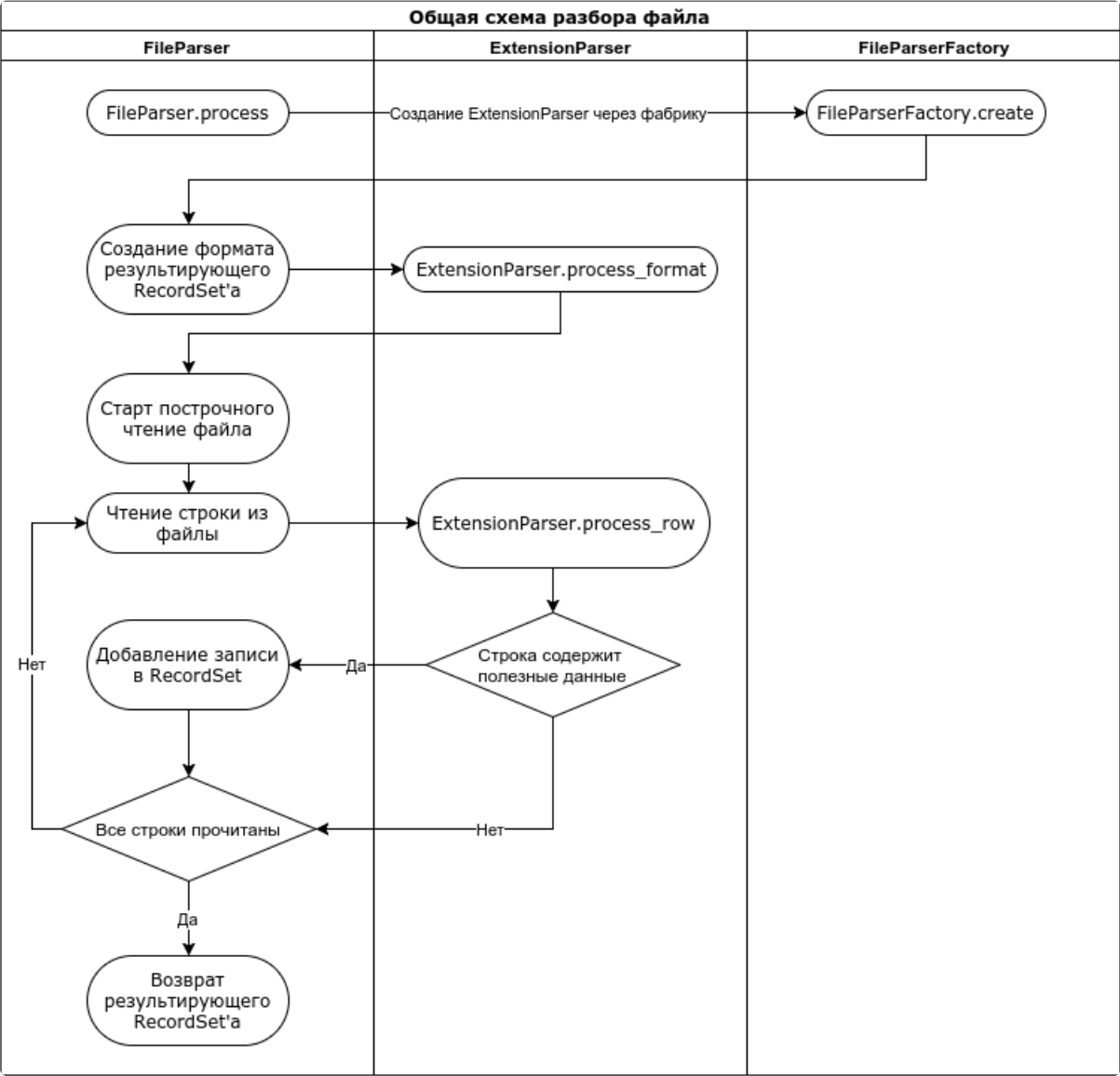


Алгоритмы и процессы

Общая схема разбора файла

Методы API предназначены для использования внутри сервиса приложения, например, при вызове из тела метода бизнес-логики. Например, в [демо-примере](#) методы API вызываются из методов с типом [Custom](#).



Построение превью документа

FileParser.create_preview(file, params)

Предразбирает файл, читает из него указанное количество строк (по умолчанию 20) и возвращает ответ в JSON-формате.

```
# Импортируем библиотеку.
import fileparser

# Создаём парсер.
parser = fileparser.FileParser()

# Разбираем первые 20 строк файла.
```

```
preview_data = parser.create_preview(file)
```

```
# Возвращаем результат в клиент.  
return preview_data
```

Метод принимает аргументы:

Название	Тип	Описание
file	sbis.RpcFile	Файл, который нужно разобрать.
params={'row':<число строк>}	int	Количество первых строк в файле, которые будут разобраны для предпросмотра.

Ответ метода возвращается в JSON-формате, поля которого описаны в следующей таблице:

Имя	Описание
file_name	Имя разобранного файла вместе с расширением. Например, "Бизнес Телеком.xlsx".
same_sheet_configs	В разобранном файле может быть несколько листов. Этот параметр оповещает, что все листы такого. По умолчанию true.
sheets	Листы разобранного файла.
sheets.index	Порядковый номер листа.
sheets.name	Имя листа.
sheets.sample_rows	Разобранные строки.
source_config	Конфигурация открытия файла. На данный момент используется только при разборе csv и txt файлов.
source_config.encoding	Название кодировки файла, например, "utf-8" или "cp1251".
source_config.delimiter	Разделитель столбцов. Используется при разборе csv и txt файлов.

На стороне клиента ответ метода преобразуется в тип [Types/source/DataSet](#), где "сырые данные" (см. метод [getRawData](#)) являются распознанными строками файла.

```
{  
  file_name: "Бизнес Телеком Импорт (раздел в отдельной колонке).xlsx",  
  same_sheet_configs: true,  
  sheets: [  
    {  
      index: 0,  
      name: "Stock&Price",  
      sample_rows: [  
        ["DK11201", "Самоклеящиеся бумажные этикетки Brother белые 29x90  
мм (400 шт.)", "Brother", "Расходные материалы/Расходные материалы QL", "2",  
null, null, "11,71", "Скидки включены", "borovkov_d@rrc.ru"]  
        ...  
      ]  
    }  
  ],  
  source_config: {  
    encoding: "utf-8",  
    delimiter: ";"  
  }  
}
```

```
}
```

Разбор документа

FileParser.process(file, params)

Разбирает файл, читает из него все строки и возвращает ответ в JSON-формате.

```
# Импортируем библиотеку.
import fileparser

# Полный разбор файла.
data = parser.process(file, Params)

# Прикладная логика по сохранению данных в БД.
...
```

Метод принимает аргументы:

Название	Тип	Описание
file	sbis.RpcFile	Файл, который нужно разобрать.
params	dict	Параметры для разбора файла. Формат поле представлен в следующей таблице.

В следующей таблице представлен формат JSON параметра params:

Имя	Описание
add_line_number	Возврат номера строки. Если в опции указано логическое значение True, то в наборах данных из листов документа будет автоматически создано целочисленное поле "__LINE__", содержащее номер строки. Нумерация строк начинается с нуля.
data_type	Имя разобранного файла вместе с расширением. Например, "Бизнес Телеком.xlsx".
destination	-
replace_all_data	Значение переключателя "В файле содержится". Когда передано значение true, пользователь выбрал значение "Все данные целиком".
file	Параметры файла.
file.name	Имя.
file.url	Ссылка.
file.uuid	UUID
same_sheet_configs	Каким образом выполняется парсинг листов. True — все листы одинаково разбираются. False — для каждого листа своя конфигурация.
sheets	Конфигурация для каждого листа файла.
sheets.index	Порядковый номер листа.
sheets.skipped_rows	Количество строк, пропускаемых парсером при разборе файла. Отсчет начинается с первой строки.
sheets.last_row	Последняя обрабатываемая строка. Отсчет начинается с первой строки. При skipped_rows == 1 и last_row == 3 будут прочитаны 2-ая и 3-я строки.

sheets.columns	Конфигурация с соответствием между полями данных и колонками.
sheets.columns.index	Числовой порядковый номер поля данных, в котором оно отображается на панели предпросмотра. Нумерация начинается с нуля.
sheets.columns.field	Имя колонки, выбранное пользователем для создания соответствия с полем данных. Список колонок задаётся в конфигурации компонента Import/Dialog в опции columns .
sheets.process_handlers	Список обработчиков импорта
sheets.process_handlers.type	Тип обработчика импорта: - applied-code-service - обработчик строк - applied-code-excel-header - обработчик реквизитов
sheets.process_handlers.id	Идентификатор обработчика
sheets.parser	Имя выбранного парсера. Например, "InColumnsHierarchyParser".
sheets.parser_config	Конфигурация парсера.
sheets.parse_config.columns	Для парсера InRowsHierarchyParser в этой опции перечислены колонки файла ровно в том порядке, в котором следует распознавать иерархию строк. Первая колонка соответствует Разделу, вторая — Подразделу и т.д. Для парсера InColumnsHierarchyParser здесь передана колонка, содержащая иерархию.
sheets.parse_config.hierarchy_name	Имя колонки, значения которой следует использовать в качестве имени иерархии. Такая колонка должна быть выбрана пользователем, чтобы парсер распознавал её данные.
sheets.parse_config.hierarchy_field	Имя поля, которое будет добавлено в формат каждой записи. По значению этого поля устанавливаются иерархические отношения между записями. Также в формат записи автоматически добавляется поле с именем имя поля@, значения которого определяют тип иерархической записи: лист, узел или скрытый узел (см. Иерархия).
sheets.parse_config.delimiter	Символ-разделитель для парсера.
source_config	Конфигурация открытия файла. На данный момент используется только при разборе csv и txt файлов.
source_config.encoding	Название кодировки файла, например, "utf-8" или "cp1251".
source_config.delimiter	Разделитель столбцов. Используется при разборе csv и txt файлов.
source_config.read_metadata	Чтение метаданных файла. True (значение по умолчанию) — при разборе необходимо вычитывать метаданные файла. False — метаданные не читаются и не учитываются при разборе. Отключение чтения метаданных существенно ускоряет разбор файлов, но при этом недоступна информация о сжатых ячейках и группировке строк (не работает OutlineHierarchyParser). При разборе больших Excel-файлов этот параметр обязательно должен быть выставлен в False.

Метод process возвращает тип list, у которого каждый элемент является типом dict.

```
>>> import fileparser
>>> parser = fileparser.FileParser()
```

```
>>> res = parser.process(file, Params)
>>> res
[{'name': 'Имя листа', 'data': <тип sbis.RecordSet>}, ...]
```

Парсеры (провайдеры разбора файла)

По умолчанию файл разбирается плоским списком "как есть". Этот функционал можно расширить с помощью специализированных парсеров. Например, через такие парсеры реализован разбор иерархии.

Библиотекой fileparser поддерживаются три типа иерархии:

- в группировке строк;
- в отдельной строке;
- в отдельной колонке.

За разбор каждого типа иерархии отвечает отдельный парсер со своими настройками. Какой именно будет использован парсер для разбора выбирает пользователь на панели ["Настройка импорта"](#).

Чтобы парсер мог разобрать иерархические строки, в его конфигурацию следует передать имя поля иерархии и имя поля, хранящее название иерархии. Как правило, такая настройка задаётся в опции providerArgs (свойства hierarchyName и hierarchyField для каждого парсера) для панели "Настройка импорта". Эти настройки используются при вызове метода [FileParser.process\(file, params\)](#) и могут быть изменены на стороне бизнес-логики.

При переключении драйвера настройки от предыдущего сохраняются.

Парсер "InColumnsHierarchyParser". Иерархия в отдельной колонке

В каждой строке в отдельной колонке указывается название раздела. Если уровней иерархии несколько, то используется несколько колонок. В примере ниже колонка "Раздел" задает иерархию 1-го уровня, а колонка "Группа" — иерархию 2-го.

	A	B	C	D
1	<u>Раздел</u>	<u>Группа</u>	<u>Наименование</u>	<u>Цена</u>
2	Memory	Flash Drive	Kingston Flash drive USB2 DatTray DTSE7/8GB KC-U768G-3PK RTL	3.70
3	Memory	Flash Drive	Kingston Flash drive USB 2.0 DataTraveler DTSE9H/8GB RTL	4.50
4	Memory	Desktop	Kingston DDR3 DIMM 2GB 1333 10660 CL9 SR x16 Rtl	15.00
5	Memory	Desktop	Kingston DDR3 DIMM 2GB 1600 12800 Single Rank X16 Rtl cl11	15.00
6	Graphics card	AGP	ATI Xpert 2000 Pro AGP4X Low Profile Bracket 32MB Original	8.00
7	Graphics card	AGP	ATI All-In-Wonder Radeon(7200) SECAM AGP4X 32MB	10.00
8	Graphics card	PCI Express	Sapphire AMD FirePRO S9170 32GB PCIE 100-505932 Retail	15.00
9	Graphics card	PCI Express	Gigabyte GV-RX16P256DE-RH X1600Pro 256MB PCIE DVI TVO Rtl	25.00

[Пример файла](#), где иерархия содержится в колонке "Группа/Класс".

Парсер "InRowsHierarchyParser". Иерархия в отдельной строке

Узлом иерархии считается отдельная строка, для которой среди всех столбцов заполнена только одна ячейка.

	A	B
1	<u>Наименование</u>	<u>Цена</u>
2	Memory	
3		Flash Drive
4	Kingston Flash drive USB2 DatTray DTSE7/8GB KC-U768G-3PK RTL	3.70
5	Kingston Flash drive USB 2.0 DataTraveler DTSE9H/8GB RTL	4.50
6		Desktop
7	Kingston DDR3 DIMM 2GB 1333 10660 CL9 SR x16 Rtl	15.00
8	Kingston DDR3 DIMM 2GB 1600 12800 Single Rank X16 Rtl cl11	15.00
9	Graphics card	
10		AGP
11	ATI Xpert 2000 Pro AGP4X Low Profile Bracket 32MB Original	8.00
12	ATI All-In-Wonder Radeon(7200) SECAM AGP4X 32MB	10.00
13		PCI Express
14	Sapphire AMD FirePRO S9170 32GB PCIE 100-505932 Retail	15.00
15	Gigabyte GV-RX16P256DE-RH X1600Pro 256MB PCIE DVI TVO Rtl	25.00

[Пример файла](#).

Для такого парсера пользователь на панели ["Настройка импорта"](#) дополнительно указывает какие из выбранных колонок соответствуют Разделу, Подразделу, Подраздел (ур. 2) и т.д.

Настройка импорта

Загрузить

В файле содержится:

Только дополнения и изменения

Все данные целиком

Загружать

Каталог

Категория

Категория товара

Раздел содержится в отдельной колонке

Начать разбор с 2 строки

	Раздел	Не задано	Наименование	Не задано
1	Группа продуктов	Подгруппа продуктов	Наименование продукта	Калорийность, ккал/100 г
2	Овощи и зелень		Артишоки	45

Парсер "OutlineHierarchyParser". Иерархия в группировке строк

Иерархия задается группировкой строк внутри Excel файла. Такая группировка отображается ввидет плюсов/минусов в служебной колонке.

1	2	3	A	B
	1		Наименование	Цена
	2		Memory	
	3			Flash Drive
	4		Kingston Flash drive USB2 DatTray DTSE7/8GB KC-U768G-3PK RTL	3.70
	5		Kingston Flash drive USB 2.0 DataTraveler DTSE9H/8GB RTL	4.50
	6			Desktop
	7		Kingston DDR3 DIMM 2GB 1333 10660 CL9 SR x16 Rtl	15.00
	8		Kingston DDR3 DIMM 2GB 1600 12800 Single Rank X16 Rtl cl11	15.00
	9		Graphics card	
	10			AGP
	11		ATI Xpert 2000 Pro AGP4X Low Profile Bracket 32MB Original	8.00
	12		ATI All-in-Wonder Radeon(7200) SECAM AGP4X 32MB	10.00
	13			PCI Express
	14		Sapphire AMD FirePRO S9170 32GB PCIE 100-505932 Retail	15.00
	15		Gigabyte GV-RX16P256DE-RH X1600Pro 256MB PCIE DVI TVO Rtl	25.00

[Пример файла.](#)

Пользовательский парсер

Вы можете создать и зарегистрировать собственный парсер для использования в собственном прикладном решении. Однако решение этой задачи выходит за рамки данной статьи.

Предполагается, что создание собственных парсеров будет выполняться опытными разработчиками под руководством ответственного сотрудника группы "Платформа" — [Абрамов В.И.](#).

Для работы с парсерами в библиотеке fileparser создана фабрика FileParserFactory. Она позволяет регистрировать и создавать парсеры по их названию. Фабрика имеет публичные методы, описанные в следующей таблице.

register(name, class_object)	Регистрирует указанный класс парсера.При совпадении имен парсеров последний зарегистрировавшийся перекрывает ранее зарегистрированные, при этом в логи пишется предупреждение.Данный метод рекомендуется вызывать при загрузке сервисных модулей из файла on_event.py .
create(name, params)	Создает экземпляр ранее зарегистрированного парсера по его имени. В конструктор будут переданы указанные в виде словаря параметры.При отсутствии парсера с указанным именем будет брошено исключение.

В парсере должны быть определены следующие методы:

process_format(format)	Данный метод вызывается для дополнения формата результирующего рекордсета. В частности парсеры, поддерживающие иерархию, должны дополнять формат полем иерархии.
process_row(row_data, record, recordset, next_row_data)	Обработать одну строку входного файла. row_data — сырые данные строки. Может использоваться, например, для опроса стилей отдельных ячеек. next_row_data — сырые данные строки, которая будет подана на разбор следующей.

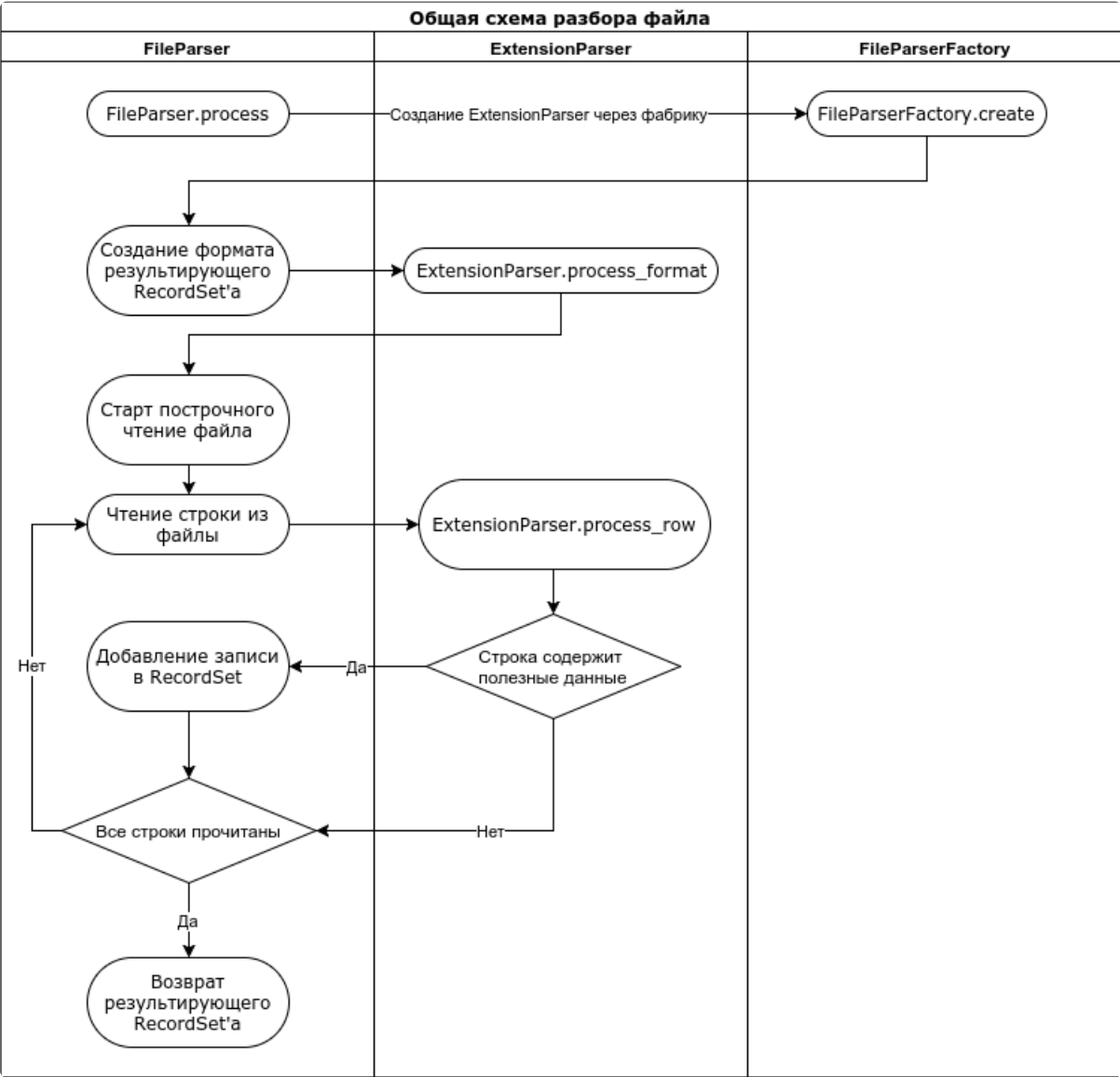
record — разобранная строка, готовая для вставки в recordset.

recordset — ссылка на результирующий recordset.

Если строка содержит "полезные" данные, то метод должен дополнить record, например, задать значение поля иерархии и вернуть True. Если метод возвращает False, то добавление строки в recordset не производится.

Кроме того, парсер может добавлять в recordset данные самостоятельно, например, записи, задающие разделы иерархии.

Схема вызовов ExtentionParser наглядно представлена на общей схеме разбора файла.



Обработчики импорта

Для работы с обработчиками импорта в библиотеке fileparser создана фабрика [FileDataHandlerRegistry](#). Она позволяет регистрировать обработчики.

Фабрика имеет публичные методы, описанные в следующей таблице:

register(handler_object)	Где handler_object - экземпляр обработчика.
type_id()	Идентификатор типа обработчика.
display_name()	Имя обработчика.
ui_editor_module()	Информация о редакторе типа обработчика.
invoke(handler_settings, raw_data, parsed_data)	Вызов обработчика, здесь handler_settings - это поле process_handler_id из параметров.

raw_data	Входной RecordSet.
parsed_data	Выходной RecordSet.

Обработчики импорта СПК

Обработчики импорта сервиса прикладного кода - операции, реализованные на языке JavaScript.

С их помощью пользователь может выполнить обработку данных документа по собственному алгоритму.

Операции СПК обработчиков выполняются специализированными парсерами, которые регистрируются в фабрике [FileDataHandlerRegistry](#).

Обработчик можно добавить в окне настройки импорта, отдельно к каждому листу Excel-документа. В режиме загрузки "Все одинаково" обработчики будут выполняться на каждом листе.

Настройка импорта для данного контрагента

Загрузить
✕

В файле содержится:

Только дополнения и изменения ☐ Все данные целиком

Настройка и загрузка листов ☒ Все одинаково ☐ Текущий ☐ Раздельно

Лист1 Лист2 Лист3

Начать разбор с **1** строки

	Не задано	Наименование	Производитель	Страна	Не задано
1	Просчет	ХОМУТ 98*2,5 ДКС 25203 100ШТ	DKC	Россия	44.69
2	Просчет	ВВОД КАБЕЛЬНЫЙ ПЛАСТИКОВЫЙ PG-29 IP68	DKC	Россия	107.96
3	Просчет	НАКОНЕЧНИК МЕДНЫЙ Т 70-12-13	IEK	Россия	49.97
4	Просчет	МУФТА ЗКВТПН-10 35-50	Нева-Транс Комплект	Россия	3023.11
5	Просчет	ХОМУТ КАБЕЛЬНЫЙ 4,8*350 100ШТ	REXANT	Беларусь	314.62
6	Просчет	ХОМУТ КАБЕЛЬНЫЙ Р6.6 2,6*160 25206 100ШТ	DKC	Россия	183.32
7	Просчет	ХОМУТ КАБЕЛЬНЫЙ 3,6*180 PLC-СВ-3,6*180	EKF	Китай	123.22
8	Просчет	МУФТА КАБЕЛЬНАЯ ЗКВТП 10 70/120	KBT	Россия	1241.09
9	Просчет	НАКОНЕЧНИК МЕДНЫЙ ТМ(О) 16-8-6	EKF	Россия	22.81
10	Просчет	НАКОНЕЧНИК МЕД М 25-10-8 -УХЛ3	ОЦМ	Россия	38.24

Обработчики

☒ Для строк
 ☐ Для реквизитов

При добавлении обработчика открывается мини-редактор прикладного кода с примером использования. При необходимости, можно открыть полнофункциональный редактор кода с помощью кнопки в углу мини-редактора.

Настройка импорта
для данного контрагента

Загрузить

В файле содержится:

Только дополнения и изменения

☒ Все данные целиком

Настройка и загрузка листов

Все одинаково

Текущий

Раздельно

Лист1

Лист2

Лист3

Начать разбор с

1

строки

	Не задано	Наименование	Производитель	Страна	Не задано
1	Просчет	ХОМУТ 98*2,5 ДКС 25203 100ШТ	ДКС	Россия	44.69
2	Просчет	ВВОД КАБЕЛЬНЫЙ ПЛАСТИКОВЫЙ PG-29 IP68	ДКС	Россия	107.96
3	Просчет	НАКОНЕЧНИК МЕДНЫЙ Т 70-12-13	IEK	Россия	49.97
4	Просчет	МУФТА ЗКВТПН-10 35-50	Нева-Транс Комплект	Россия	3023.11
5	Просчет	ХОМУТ КАБЕЛЬНЫЙ 4,8*350 100ШТ	REXANT	Беларусь	314.62
6	Просчет	ХОМУТ КАБЕЛЬНЫЙ Р6.6 2,6*160 25206 100ШТ	ДКС	Россия	183.32
7	Просчет	ХОМУТ КАБЕЛЬНЫЙ 3,6*180 PLC-СВ-3,6*180	EKF	Китай	123.22
8	Просчет	МУФТА КАБЕЛЬНАЯ ЗКВТП 10 70/120	КВТ	Россия	1241.00
9	Просчет	НАКОНЕЧНИК МЕДНЫЙ ТМ(О) 16-8-6	EKF	Россия	22.81
10	Просчет	НАКОНЕЧНИК МЕД М 25-10-8 -УХЛ3	ОЦМ	Россия	38.24

Обработчики

Для реквизитов

Для строк

1

// Выход.КолонкаРеестра = Вход.КолонкаВФайле

×

Обработчики импорта строк

Предназначены для дополнения и корректировки данных в строках документа.

Выполняются после чтения данных листа. Для каждой строки документа обработчик вызывается отдельно.

Исполнитель обработчика строк регистрируется с идентификатором `applied-code-service`.

В контексте выполнения операции находятся объекты `Вход` и `Выход`.

Объект `Вход` содержит данные строки импортируемого документа. Поля объекта `Вход` совпадают с колонками в файле и именуются в соответствии с индексом столбца, в формате `fld_x`, где `x` - индекс столбца, начиная с нуля. Например, в поле `fld_0` содержится значение первого столбца, в поле `fld_15` - шестнадцатого.

Объект `Выход` - целевой объект обработчика, содержит вычитанные из этой же строки данные. Поля объекта `Выход` совпадают по названиям с колонками реестра. Модифицированные обработчиком данные объекта будут возвращены для записи в БД.

Обработчики реквизитов

Предназначены для извлечения из документа реквизитов и другой информации, размещенной в шапке документа, перед строками данных.

Выполняются перед чтением данных листа. Для каждого листа обработчик вызывается однократно, если указан в настройках импорта.

Исполнитель обработчика реквизитов регистрируется с идентификатором `applied-code-excel-header`.

В контексте выполнения операции находятся объекты `Строки` и `Реквизиты`, а также функция `Ячейка`.

Объект `Строки` содержит данные строк импортируемого документа до начала табличных данных и представляет собой массив объектов. Поля объекта совпадают с колонками в файле и именуются в соответствии с индексом столбца, в формате `fld_x`, где `x` - индекс столбца, начиная с нуля. Например, в поле `fld_0` содержится значение первого столбца, в поле `fld_15` - шестнадцатого.

Объект `Реквизиты` - целевой объект обработчика, содержит поля, совпадающие по названиям с реквизитами документа. Модифицированные обработчиком данные объекта будут возвращены для записи в БД.

Функция `Ячейка` предназначена для удобного доступа к ячейке таблицы, принимает строку с адресом в формате Excel, возвращает строку со значением в ячейке с указанным адресом.

В результате выполнения метода `FileParser.process` данные реквизитов возвращаются в Record `metadata` в данных листов. Ниже приведен пример доступа к полученным реквизитам:

```

1 parser = fileparser.FileParser()
2 parsed_data = parser.process(rpc_file, template)

```

```

3  for sheet_data in parsed_data:
4      # получение данных листа, в формате RecordSet
5      rows_data = sheet_data['data']
6      # обработка строк данных
7      # ...
8
9      # получение реквизитов листа, в формате Record
10     requisites = sheet_data['metadata']
11     # обработка реквизитов
12     # ...

```

Хранение обработчиков импорта

При создании обработчика импорта, код обработчика сохраняется на [сервисе прикладного кода](#). Контекст СПК для обработчика определяется по его типу и пространству имен импорта вызовом метода

`ImportAssistant.GetImportHandlerContext` с сервиса [import-assistant](#). Список актуальных контекстов опубликован в разделе БЗ [Контексты обработчиков импорта](#).

При выполнении импорта без использования шаблона (например, если после настройки импорта не согласиться на предложение сохранить шаблон) обработчики импорта создаются временно. После завершения выполнения импорта такие обработчики будут удалены с сервиса прикладного кода и не сохранятся для повторного использования. Временные шаблоны определяются по пустому полю `template_id`.

Данные операции сохраняются в шаблоне импорта в блоке описания листа. Для этого создается новая запись в списке `process_handlers`, в поле `id` записывается GUID обработчика, в поле `type` - его тип.

Полный формат шаблона и пример приведены в разделе БЗ [Формат шаблона импорта](#)

Редактирование и исполнение прикладного кода

Изменение кода обработчика выполняется редактором СПК. К прикладному коду обработчика применяются стандартные правила линтера СПК. Дополнительно проверяется, что в коде есть изменения полей целевого объекта обработчика (то есть объекта **Выход** для обработчика строк и **Реквизиты** для обработчика реквизитов).

Права на чтение и изменение обработчиков определяются по зонам реестра, в котором выполняется импорт. Зоны доступа приведены в таблицах ["Контексты СПК обработчиков импорта"](#) и ["Контексты СПК обработчиков реквизитов"](#)

Статистика использования обработчиков импорта

Статистика импорта с использованием обработчиков импорта СПК, а также их создания и изменения публикуется в прикладной статистике по стандартной схеме хранения Функционал-Контекст-Действие.

Используется функционал "Импорт из Excel", в качестве контекста используется системное имя контекста СПК.

Сохраняются действия:

- Импорт с обработчиком
- Создание обработчика импорта
- Изменение обработчика импорта

Событие импорта с обработчиком публикуется из прикладных методов импорта данных платформенным инструментом `FileParser`, при инициализации чтения Excel-документа в методе `process`.

События создания и изменения обработчика импорта публикуются на сервисе СПК при операциях с кодом обработчика.

Функционал (Топ 100): Импорт из Excel	Контекст (Топ 100):	Количество вызовов	Количество уникальных клиентов	Количество уникальных пользователей
Импорт из Excel		72 742	3 793	4 196
WarehouseDocImport		18 835	1 043	1 152
Своя импорт		10 102	1 034	1 543
Импорт по шаблону		5 953	354	602
Создание шаблона импорта		551	258	263
Удаление шаблона импорта		172	41	41
Изменение шаблона импорта		52	32	33
Создание обработчика импорта		3	3	3
Изменение обработчика импорта		1	1	1
Импорт с обработчиком		1	1	1
MoneyUploadImport		11 510	159	171
Excel		10 681	1 046	1 141
NomenclatureImportProcessHandler		9 288	1 422	1 489
ParseExcelDocImport		5 173	200	233
CommissionChangeImport		5 142	170	172
ParseExcelPriceMatchingImport		2 530	213	222
OfPaymentsImport		1 364	57	59
NotfoundNamespacesImport		1 358	37	72
ParseExcelCenImport		1 352	63	75
DiscountCardImport		1 300	32	34
ParseExcelZakazImport		893	16	17
WholesaleImport		790	179	184
PriceImport		787	42	54
ParseExcelAnHueImport		699	47	52
StaffListImport		589	135	141
FundLoadTrpImport		379	2	5
AccountingImport		367	11	11

Рисунок 1. Прикладная статистика импорта из Excel

Указанные метрики отображаются в [статистике использования системы](#), в разделе Базовые возможности/Загрузка из табличных документов.

Наименование	Статус	Активность	Аккаунты	Активность	Аккаунты	Новые	Роль	Δ%	Вс
Загрузка из табличных файлов (Excel)	3 105								
Клиентские документы	0	0	133	8	8	0	+100	доп	
Клиенты CRM	0	0	302	69	69	0	+100	доп	
Компании	0	0	0	0	0	0	0	доп	
Каталоги	0	0	0	0	0	0	0	доп	
Наличие	0	0	0	0	0	0	0	доп	
Настройка выгрузки	0	0	139	19	19	0	+100	доп	
Начисления сотрудников	0	0	0	0	0	0	0	доп	
Номенклатура	0	0	9 208	1 236	1 216	0	+100	доп	
Импорт	0	0	8 497	1 206	1 206	0	+100	доп	
Импорт по шаблону	0	0	2 003	307	307	0	+100	доп	
Импорт с обработчиком	0	0	1	1	1	0	+100	доп	
Своя импорт	0	0	6 493	1 206	1 206	0	+100	доп	
Обработка	0	0	6	4	4	0	+100	доп	
Изменение обработки импорта	0	0	0	0	0	0	0	доп	
Создание обработки импорта	0	0	6	4	4	0	+100	доп	
Удаление обработки импорта	0	0	0	0	0	0	0	доп	
Шаблон	0	0	799	320	310	0	+100	доп	
Изменение шаблона импорта	0	0	35	18	18	0	+100	доп	
Создание шаблона импорта	0	0	661	293	293	0	+100	доп	
Удаление шаблона импорта	0	0	63	43	43	0	+100	доп	
Обработка	0	0	18	4	4	0	+100	доп	
Остатки имущества	0	0	26	8	8	0	+100	доп	
Остатки по отпускам	0	0	30	3	3	0	+100	доп	
Отправка инструкций	0	0	0	0	0	0	0	доп	
Отчеты дирекции дивизиона	0	0	365	11	11	0	+100	доп	

Рисунок 2. Статистика использования обработчиков импорта СПК

Шаблоны импорта

Шаблон импорта используется для структуризации информации, связанной с настройками импорта.

Применяется в подборе подходящего шаблона импорта, в работе со статистикой позиционирования полей в шаблоне, является настройками, передаваемыми в поле params для разбора файла в методе FileParser.process.

Методы БЛ для работы с шаблонами:

- ImportTemplate.Create
- ImportTemplate.Read
- ImportTemplate.Update
- ImportTemplate.Delete

- ImportTemplate.List

Формат шаблона импорта

Ключ	Описание
name	Название шаблона
fileExtension	Тип файла
file	Объект с описанием файла, опциональный.
file/name	Имя файла
template_namespace	Только у инсталляционных шаблонов. Имя пространства имен.
template_version	Только у инсталляционных шаблонов. Версия включения.
template_uuid	Только у инсталляционных шаблонов. UUID шаблона.
columnsSeparator	Разделители столбцов
same_sheet_configs	Использовать ли общие настройки для всех листов
sheets	Список листов
sheets/index	Номер листа. Отсутствует, если настройка для всех листов
sheets/skipped_rows	Сколько строк пропустить
sheets/columns	Список столбцов
sheets/columns/index	Номер столбца, нумерация с 0
sheets/columns/field	Название столбца
sheets/hierarchyParser	Объект с описанием иерархии для парсера
sheets/document_format	Опциональный. Описание полей для сверки формата документа.
sheets/document_format/line	Номер строки, нумерация с 0
sheets/document_format/format	Формат строки
sheets/document_format/format/index	Номер столбца, нумерация с 0
sheets/document_format/format/field	Точное содержимое строки
sheets/process_handlers	Опциональный. Список обработчиков импорта.
sheets/process_handlers/id	Идентификатор обработчика
sheets/process_handlers/type	Тип обработчика: • applied-code-service - обработчик строк • applied-code-excel-header - обработчик шапки
sheets/process_handlers/fields	Только для обработчика шапки. Список поддерживаемых полей.
parse_as_json	Разобрать как json
replace_all_data	Заменить все данные

Пример результата в формате JSON:

```
1 {
2   "fileExtension": "excel",
3   "file":
4   {
```

```

5         "name": "загрузка заказов т2.xlsx"
6     },
7     "columnsSeparator": null,
8     "sheets":
9     [
10        {
11            "columns":
12            [
13                {
14                    "index": 0,
15                    "field": "Наименование"
16                },
17                {
18                    "index": 1,
19                    "field": "Артикул"
20                }
21            ],
22            "skippedRows": 0,
23            "hierarchyParser": {}
24        }
25    ],
26    "sameSheetConfigs": true,
27    "parse_as_json": false,
28    "replace_all_data": true,
29    "name": "111222"
30 }

```

Обработчики данных аккаунта для шаблонов импорта

Обработчик удаления данных аккаунта

Обработчик удаления пользовательских шаблонов импорта реализован в методе БЛ

DeregPersonalDataClientHdl.ImportTemplateClearClient, в составе служебного модуля **FileParserTemplateHandler**.

Вышеуказанный модуль входит в состав шаблонного сервис **online32**. Вызов метода выполняется платформенным механизмом в модуле **Account Data**, входящий в список зависимостей модуля **FileParserTemplateHandler**

Обработчик рециркуляции аккаунтов

Обработчик рециркуляции пользовательских шаблонов импорта реализован в методе БЛ

RegRecoveryClientDataHdl.CopyClientTemplates, в составе служебного модуля **FileParserTemplateHandler**. Вызов метода выполняется платформенным механизмом по имени объекта БЛ **RegRecoveryClientDataHdl**.